API Request Validation

Pydantic, 422, unknown fields and size limits

Watch the 4-min video

In one line: validate in layers, as early and cheaply as possible: coarse size and type limits at the proxy (413), schema rules at the service edge (422), and business rules in the domain layer (409); reject unknown fields, use strict types for money and IDs, and cap every size.

1 A request that should never reach your code

A signup request arrives. The email is `bob@nowhere`, the age is **−5**, the name is **50 megabytes** long, and there's an extra field: `is_admin: true`. Every one of these should be stopped **before it touches your business logic**. That's request validation.

```
{
"email": "bob@nowhere",
"age": -5,
"name": "AAAA… (50 MB)",
"is_admin": true
}
```

---- EmailStr: not a valid email domain — **schema → 422**

---- age must be 13 to 120 — **schema → 422**

---- body over 1 MB, stopped at the proxy — **proxy → 413**

---- unknown field, extra='forbid' — **schema → 422**

Figure 1. Four problems in one body. Each is caught by a different rule; the oversized name never even reaches the app because the proxy rejects the body.

2 Validate in layers, as early as possible

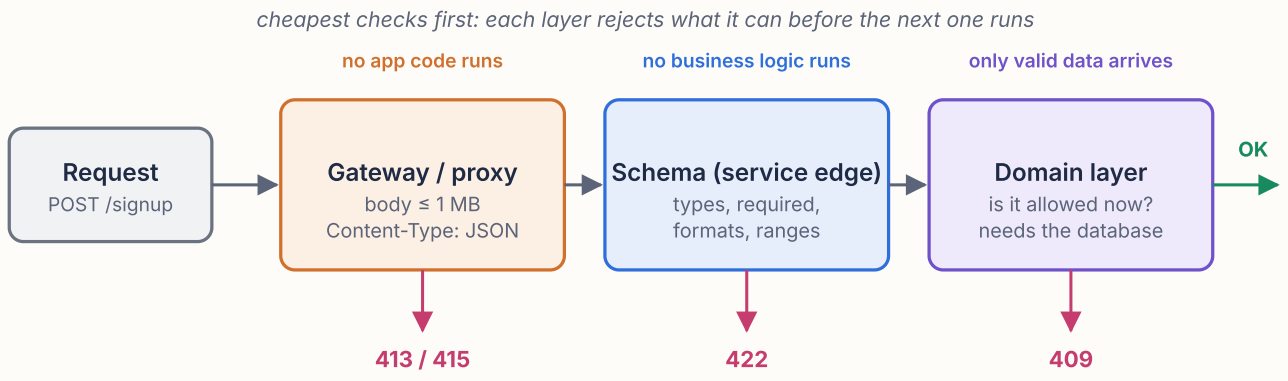


Figure 2. Each layer rejects what it can, as cheaply as it can. By the time a request reaches the domain layer, its shape is already guaranteed.

- **Gateway or reverse proxy:** coarse limits like body size and content type. Too big → `413 Content Too Large`; wrong type → `415 Unsupported Media Type`.
- **Service edge (schema):** required fields, types, formats and ranges. Fails → `422` with a list of which fields failed and why.

- **Domain layer:** business rules. Is this allowed right now? These usually need the database.

Client-side checks in the app or browser are nice for the user, but they are not security: anyone can send raw HTTP. **The server is the real boundary.**

3 Schema validation with Pydantic (FastAPI)

In FastAPI you declare a Pydantic model and use it as the request body type. FastAPI validates every request against it before your function runs:

```
from pydantic import BaseModel, ConfigDict, EmailStr, Field

class SignupRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")        # unknown fields → error
    email: EmailStr                                   # needs the email-validator package
    age: int = Field(ge=13, le=120)
    name: str = Field(min_length=1, max_length=100)
    tags: list[str] = Field(default=[], max_length=20)

@app.post("/signup", status_code=201)
def signup(req: SignupRequest): ...
```

If a request doesn't match, FastAPI returns **422** automatically, listing exactly which fields failed:

```
HTTP/1.1 422 Unprocessable Entity
{"detail": [
  {"type": "greater_than_equal", "loc": ["body", "age"],
   "msg": "Input should be greater than or equal to 13", "input": -5},
  {"type": "extra_forbidden", "loc": ["body", "is_admin"],
   "msg": "Extra inputs are not permitted", "input": true}
]}
```

Other stacks do the same with **JSON Schema** or **OpenAPI** validators (for example Ajv in Node.js, or validating at the API gateway against the OpenAPI spec).

4 Unknown fields and type coercion

Reject unknown fields. With `extra="forbid"`, the sneaky `is_admin` field fails validation instead of being silently accepted. This blocks **mass assignment** attacks, where a client sets a field you never meant to expose (for example, when code copies the request body straight into a database row). It also catches client typos early: `emial` fails loudly instead of being ignored. (Pydantic's default is to *ignore* extra fields, so you must opt in.)

Be careful with coercion. Lenient ("lax") parsing turns the string `"42"` into the integer 42, which is convenient. But it can also turn the string `"1"` into `True`, or accept a float like `19.999999` where you need an exact amount. For **money and IDs, use strict types**, so the value must already have the right type: `StrictInt`, `Field(strict=True)`, or integer cents / `Decimal` for amounts.

5 Oversized payloads

- **Cap the body at the proxy**, for example 1 MB with nginx's `client_max_body_size 1m;`. Huge bodies are rejected with **413** before your app even reads them.
- **Cap inside the schema:** maximum string lengths, maximum array items and maximum nesting depth. A body under 1 MB can still hold a 200,000-element array that costs a lot of CPU and memory to parse, validate and process.
- **Large files skip the API entirely** and upload directly to object storage with a presigned URL.

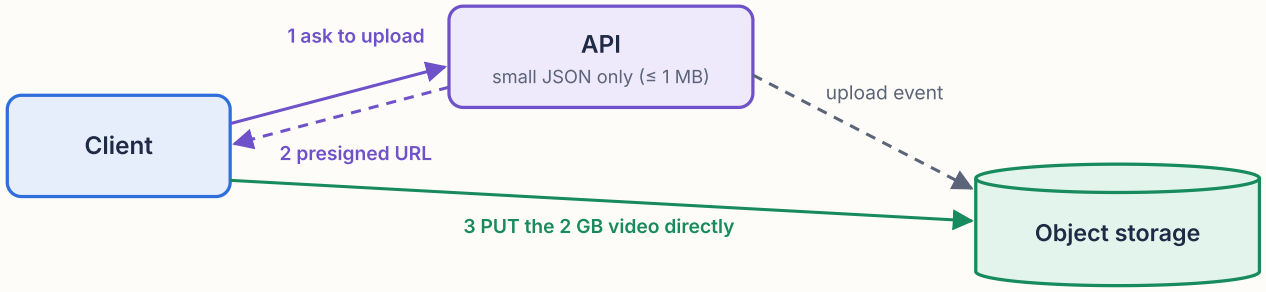


Figure 3. The API only hands out a short-lived presigned URL; the large file goes straight to object storage, which then notifies the backend.

WORKED EXAMPLE

Without limits, an attacker sends 100 requests per second, each a 50 MB name. That's **5 GB/s** of data your app servers must receive, buffer and parse: a few seconds are enough to exhaust memory. With `client_max_body_size 1m`, nginx answers 413 as soon as it sees the Content-Length header (or once a streamed body passes 1 MB), and the app never sees the requests.

Inside the limit: `max_length=100` on name and `max_length=20` on tags means one valid body can't exceed a few kilobytes of meaningful data.

6 Validation vs business rules

	Validation (schema)	Business rules (domain)
Question	Is this request well formed?	Is this allowed right now?
Examples	Is the email a valid email? Is age 13–120?	Is the email already registered? Is there enough balance?
Needs the database?	No	Usually yes
Where	Service edge (Pydantic, JSON Schema)	Domain / service layer
Typical error	`422` with field errors	`409` with a specific code, e.g. `EMAIL_TAKEN`

COMMON MISTAKES

- Trusting client-side validation as security.
- Leaving extra fields allowed (Pydantic's default) and copying the body into a model: mass assignment.
- Lax coercion for money and IDs, so `"1"` becomes `True` or a float slips in.
- No body size limit at the proxy, and no `max_length` on strings and arrays.
- Putting database lookups in the schema validators, or returning 422 for “email already taken”.

7 Key terms

Schema validation: types, required fields, formats, ranges

extra="forbid": unknown fields fail validation

422 Unprocessable Entity: well-formed JSON, invalid fields

Mass assignment: client sets fields it shouldn't

413 Content Too Large: body exceeds the size limit

Type coercion: lax conversion, e.g. "42" → 42

Strict mode: value must already have the type

Presigned URL: short-lived direct upload link

Business rule: context check, often needs the DB

8 Interview questions

Q1. Where should validation happen? BASIC

In layers: coarse limits at the gateway (body size, content type), schema validation at the service edge with models like Pydantic (422), and business rules in the domain layer (409). Cheapest checks first.

Q2. How do you protect against oversized payloads? BASIC

Cap the body at the proxy (e.g. 1 MB in nginx) so it returns 413 before the app reads it; limit string lengths, array sizes and nesting depth in the schema; send large files directly to object storage with presigned URLs.

Q3. 400 vs 422 vs 409? BASIC

400: the request is malformed or can't be parsed (FastAPI reports bad JSON as 422 too, by default). 422: valid JSON but fields fail validation (age = −5). 409: valid, but conflicts with current state (email already registered).

Q4. A user sends is_admin: true in a signup body. What stops it? SCENARIO

extra="forbid" on the request model makes it a 422. Also keep separate input and database models so even an allowed field can't map to is_admin.

Q5. A client sends amount_cents: "1000" (a string) and a flag as "1", and both are accepted. Problem? SCENARIO

Lax coercion turned them into 1000 and True. For money, IDs and flags use strict types (StrictInt, StrictBool or Field(strict=True)) so wrong types are rejected with 422, and the client fixes its bug instead of relying on guesses.

Q6. A 900 KB request with a 150,000-item array spikes CPU. The proxy limit is 1 MB. Why? SCENARIO

The body passes the size limit, but nothing caps the array. Add max_length on lists (e.g. 100 items) and limits on nesting depth, so validation fails fast.

Q7. Should “email already registered” be checked in a Pydantic validator? TRICKY

No. It needs the database and can change between check and insert. Check it in the domain layer, enforce it with a unique constraint, and return 409 EMAIL_TAKEN.

Next: SD-C-011 · API Error Handling backendroadmap.com